KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

# COLLEGE EVENT MANAGEMENT SYSTEM

Project Report submitted in partial fulfillment of the requirements for the award of the degree of

## BACHELOR OF SCIENCE (BSc)

**Computer Science, Mathematics**


**Submitted by**

Harshitha Reddy.W.S

ROLL NO:26CSMM19

Under the guidance of

Ms. JOMOL R.K

SCHOOL OF COMPUTATIONAL AND PHYSICAL SCIENCES
DEPARTMENT OF PHYSICAL SCIENCES
KRISTU JAYANTI (Deemed to be University)
K. Narayanapura, Kothanur P.O., Bangalore – 560077

KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

**DEPARTMENT OF PHYSICAL SCIENCES**

# CERTIFICATE OF COMPLETION

This is to certify that the project entitled **College Event Management System** has been satisfactorily completed by **Harshitha Reddy.W.S** in partial fulfillment of the award of the Bachelor of Science (Computer Science, Mathematics) degree requirements prescribed by Kristu Jayanti (Deemed to be University), Bengaluru (Affiliated to Bengaluru North University) during the academic year 2026-2027.

***Internal Guide*** ***Head of the Department***

***Valued by Examiners***

**1:_____** ***Centre:*** **Kristu Jayanti(Deemed to be University)**
**2:_____** ***Date:***

KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

## DECLARATION

I, **Harshitha Reddy.W.S** hereby declare that the project work entitled College Event Management System is an original project work carried out by me, under the guidance of Ms.Jomol R.K.

This project work has not been submitted earlier either to any University / Institution or any other body for the fulfillment of the requirement of a course of study.

Signature

**Harshitha Reddy.W.S**

**26CSMM19**

Bengaluru

Date:

# ACKNOWLEDGEMENT

The success of the project depends upon the efforts invested. It's my duty to acknowledge and thank the individuals who have contributed to the successful completion of the project.

I take this opportunity to express my profound and whole-hearted thanks to **Rev. Fr. Dr. AUGUSTINE GEORGE, VICE CHANCELLOR** and **Rev. Fr. Dr. LIJO P THOMAS, PRO-VICE CHANCELLOR**, KRISTU JAYANTI (DEEMED TO BE UNIVERSITY), BENGALURU, for providing ample facilities to undertake my project successfully.

I express my deep sense of gratitude and sincere thanks to our **Dean, Dr. SEVUGA PANDIAN A**, Dean, School of Computational and Physical Sciences, and **Head, Dr. Liji George**, Head of the Department, Department of Physical Sciences, for their valuable advice.

I feel immense pleasure to thank my respected guide **Ms.JOMOL** R.K for sustaining interest and providing dynamic guidance in aiding me to complete this project immaculately and impeccably and for being the source of my strength and confidence.

It is my duty to express my thanks to all teaching and non-teaching staff members of the department who offered me help directly or indirectly by their suggestions. The successful completion of my project would not have been possible without my parents' sacrifice, guidance, and prayers. I take this opportunity to thank everyone for their continuous encouragement. I convey my thankfulness to all my friends who were with me to share my happiness and agony.

Last but not the least I thank the Almighty God for giving me strength and good health throughout my project and enabling me to complete it successfully.

# SYNOPSIS

The **College Event Management System** is a web-based application developed to simplify the process of managing and participating in college events. Colleges organize various events such as workshops, seminars, competitions and cultural programmes. Managing event information and student registrations manually can be time-consuming.

The proposed system provides an organized platform where students can register, log in, view available college events and participate in events through an online registration process.

The application is developed using HTML, CSS and JavaScript for the front-end, Node.js and Express.js for the back-end, and MongoDB for database management.

The main features of the system include student registration, secure login, event display through event cards, joining events and viewing the events in which a student has registered.

The system aims to reduce manual work, improve event information accessibility and provide a convenient way for students to participate in college activities.

# TABLE OF CONTENTS

**1.0 INTRODUCTION** 1
- **1.1 Modules in the Project** 1
  - **1.1.1 User Authentication Module** 1
  - **1.1.2 Event Display Module** 1
  - **1.1.3 Event Registration Module** 1
  - **1.1.4 My Events Module** 1

**2.0 SYSTEM CONFIGURATION** 1
- **2.1 Hardware Requirements** 1
- **2.2 Software Requirements** 1

**3.0 DETAILS OF SOFTWARE** 2
- **3.1 Overview of Front-End** 2
- **3.2 Overview of Back-End** 2

**4.0 MODULE IMPLEMENTATION** 2
- **4.1 User Authentication Module** 2
  - **4.1.1 HTML Code** 2
  - **4.1.2 CSS Code** 4
  - **4.1.3 JavaScript Code** 8
  - **4.1.4 Node.js Code** 13
  - **4.1.5 Output Screenshot** 21
- **4.2 Event Display Module** 23
  - **4.2.1 HTML Code** 24
  - **4.2.2 CSS Code** 24
  - **4.2.3 JavaScript Code** 24
  - **4.2.4 Node.js Code** 25
  - **4.2.5 Output Screenshot** 25
- **4.3 Event Registration Module** 26
  - **4.3.1 HTML Code** 26
  - **4.3.2 CSS Code** 27
  - **4.3.3 JavaScript Code** 27
  - **4.3.4 Node.js Code** 27
  - **4.3.5 Output Screenshot** 27
- **4.4 My Events Module** 29
  - **4.4.1 HTML Code** 29
  - **4.4.2 CSS Code** 30
  - **4.4.3 JavaScript Code** 30
  - **4.4.4 Node.js Code** 30
  - **4.4.5 Output Screenshot** 30

**5.0 DATABASE** 31

**5.1 Database Design (Table Structure) 31**
**5.2 Database Output Screenshot 32**
**6.0 VALIDATION 33**
**6.1 Form Validation (Front-End) 33**
**6.2 Backend Validation (Node.js) 33**
**6.3 Validation Output Screenshot 33**
**7.0 CONCLUSION 37**
**8.0 REFERENCES 37**

# 1.0 INTRODUCTION

The **College Event Management System** is a web-based application developed to simplify the process of managing and participating in college events. Colleges regularly organize technical events, cultural programmes, workshops, seminars, sports competitions, and other extracurricular activities. Managing event information and student participation manually can be time-consuming and prone to errors.

The proposed system provides a centralized platform where students can register, log in, view available events, and participate through an online registration process. The application maintains event details and participant records in a MongoDB database, ensuring secure and organized data management.

The system is developed using HTML, CSS, and JavaScript for the front-end, Node.js and Express.js for the back-end, and MongoDB for database management. It helps improve communication, reduce paperwork, and encourage student participation in college activities

## 1.1 MODULES IN THE PROJECT

### 1.1.1 User Authentication Module

This module allows students to register and log in to the system securely. Passwords are encrypted using bcryptjs before they are stored in the database. JSON Web Tokens (JWT) are used to authenticate users and protect access to authorized pages.

### 1.1.2 Event Display Module

This module displays all available college events. Students can view event details such as title, description, category, date, and venue. The information is dynamically retrieved from the MongoDB database and presented through event cards.

### 1.1.3 Event Registration Module

This module enables students to register for events. After selecting an event, the student can join it with a single click. The system stores the registration details and prevents duplicate registrations for the same event.

### 1.1.4 My Events Module

This module allows students to view all the events they have joined. Registered events are displayed in a separate page with their details, helping students track their participation in college activities.

# 2.0 SYSTEM CONFIGURATION

## 2.1 HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 500 MB free space
- Monitor: Standard display
- Keyboard and Mouse
- Internet connection for installation and development resources

## 2.2 SOFTWARE REQUIREMENTS

- Operating System: Windows 10 or above
- Front-End: HTML, CSS and JavaScript
- Back-End: Node.js
- Framework: Express.js

- Database: MongoDB
- Code Editor: Visual Studio Code
- Browser: Google Chrome or Microsoft Edge
- Packages: Mongoose, bcryptjs, jsonwebtoken and dotenv

## 3.0 DETAILS OF SOFTWARE

### 3.1 OVERVIEW OF FRONT-END

The front-end of the College Event Management System is developed using HTML, CSS, and JavaScript.

**HTML:** HTML is used to create the structure of the application, including registration forms, login forms, event pages, and student dashboards.

**CSS:** CSS is used to design the user interface by providing colours, layouts, responsiveness, spacing, and styling for forms and event cards.

**JavaScript:** JavaScript is used for handling user interactions, form submission, event registration, authentication, and communication with the back-end APIs.

### 3.2 OVERVIEW OF BACK-END

The back-end of the College Event Management System is developed using Node.js and Express.js.

**Node.js:** Provides the runtime environment for executing server-side JavaScript.

**Express.js:** Used for developing APIs related to student registration, login, event retrieval, and event registration.

**MongoDB:** Stores student, event, and registration data.

**Mongoose:** Provides schema-based interaction with MongoDB.

**bcryptjs:** Encrypts passwords before storing them in the database.

**JSON Web Token (JWT):** Provides secure authentication and authorization for users.

**dotenv:** Stores environment variables such as database connection strings and secret keys securely

## 4.0 MODULE IMPLEMENTATION

### 4.1 USER AUTHENTICATION MODULE

User Authentication Module provides registration and login functionality for students. Students can create an account by providing their personal details and log in using their registered email address and password. Passwords are securely stored using bcrypt hashing, and JWT authentication is used to protect authorized pages.

#### 4.1.1 HTML CODE

The registration and login pages are created using HTML forms. These forms collect the user's name, email address and password.

Insert the following project files:

- login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Login</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
    <div class="auth-container">
        <h1>🎓 CampusConnect</h1>
        <h2>Welcome Back!</h2>
        <p>Login to discover upcoming college events.</p>
        <form id="loginForm">
            <input type="email" id="email" placeholder="Email Address" required>
            <input type="password" id="password" placeholder="Password" required>
            <button type="submit">Login</button>
        </form>
        <p id="message"></p>
        <p>New student? <a href="register.html">Create an account</a></p>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

- register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Registration</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="auth-container">
        <h1>🎓 CampusConnect</h1>
        <h2>Student Registration</h2>
        <p>Create your account to explore college events.</p>
        <form id="registerForm">
            <input type="text" id="name" placeholder="Full Name" required>
            <input type="email" id="email" placeholder="Email Address" required>
            <select id="department" required>
                <option value="">Select Department</option>
                <option>Computer Science</option>
                <option>Information Technology</option>
                <option>Electronics</option>
                <option>Mechanical</option>
                <option>Civil</option>
                <option>Commerce</option>
                <option>Arts and Science</option>
            </select>
            <select id="year" required>
                <option value="">Select Year</option>
                <option>1st Year</option>
                <option>2nd Year</option>
                <option>3rd Year</option>
                <option>4th Year</option>
            </select>
            <input type="password" id="password"
                placeholder="Create Password (minimum 8 characters)"
                minlength="8" required>
            <button type="submit">Create Account</button>
        </form>
        <p id="message"></p>
        <p>Already registered? <a href="login.html">Login here</a></p>
    </div>
    <script src="script.js"></script>
```

```
</body>
</html>
```

**4.1.2 CSS CODE**

CSS is used to style the registration and login forms, including input fields, buttons, headings and page layouts.

style.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: #f4f3fb;
    color: #25213a;
}
.auth-container {
    width: 90%;
    max-width: 420px;
    margin: 55px auto;
    padding: 35px;
    background: white;
    border-radius: 16px;
    box-shadow: 0 5px 25px rgba(30, 20, 60, 0.12);
    text-align: center;
}
.auth-container h1 {
    color: #7045c6;
    margin-bottom: 20px;
}
.auth-container h2 {
    margin-bottom: 12px;
}
.auth-container p {
    margin: 15px 0;
    color: #777;
    line-height: 1.5;
}
```

```
input, select {
    width: 100%;
    padding: 13px;
    margin: 9px 0;
    border: 1px solid #d8d1e8;
    border-radius: 7px;
    font-size: 14px;
    background: white;
}

button {
    padding: 12px 18px;
    border: none;
    border-radius: 7px;
    background: #7045c6;
    color: white;
    font-size: 14px;
    cursor: pointer;
}
button:hover {
    background: #5732a3;
}
.auth-container button {
    width: 100%;
    margin-top: 12px;
}

a {
    color: #7045c6;
    text-decoration: none;
}
a:hover {
    text-decoration: underline;
}

nav {
    background: #211638;
    color: white;
    padding: 18px 6%;
```

```
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 15px;
    flex-wrap: wrap;
}

nav h2 {
    color: #d8c2ff;
}

nav a {
    color: white;
    margin: 0 12px;
}

nav button {
    background: #a16af0;
}
.hero {
    text-align: center;
    padding: 65px 20px;
    background: linear-gradient(120deg, #5732a3, #9570dc);
    color: white;
}
.hero h1 {
    font-size: 36px;
    margin-bottom: 15px;
}
.hero p {
    font-size: 16px;
}
.hero h3 {
    margin-top: 18px;
}
.main-container {
    width: 88%;
    max-width: 1200px;
    margin: 40px auto;
```

```
}
.main-container h2 {
    margin-bottom: 25px;
}
.event-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 24px;
}
.event-card {
    background: white;
    border-radius: 12px;
    padding: 25px;
    box-shadow: 0 4px 15px rgba(30, 20, 60, 0.08);
    transition: transform 0.2s;
}
.event-card:hover {
    transform: translateY(-5px);
}
.event-card h3 {
    color: #7045c6;
    margin: 12px 0;
}
.event-card p {
    color: #666;
    margin: 10px 0;
    line-height: 1.5;
}
.category {
    display: inline-block;
    padding: 6px 12px;
    background: #eee6ff;
    color: #7045c6;
    border-radius: 20px;
    font-size: 12px;
}
.event-card button {
    width: 100%;
    margin-top: 15px;
```

```
}
.event-card button:disabled {
    background: #999;
    cursor: not-allowed;
}
#message {
    text-align: center;
    margin: 20px 0;
    font-weight: bold;
}
@media (max-width: 600px) {
    .hero h1 {
        font-size: 27px;
    }

    nav {
        flex-direction: column;
    }

    nav a {
        display: inline-block;
        margin: 8px;
    }
    .main-container {
        width: 92%;
    }
}
```

### 4.1.3 JAVASCRIPT CODE

JavaScript handles form submission and sends registration and login information to the server using the Fetch API.

It also stores the authentication token after successful login.

- script.js

```
document.addEventListener("DOMContentLoaded", () => {
    const token = localStorage.getItem("token");
    const page = document.body.dataset.page;
    const message = document.getElementById("message");
    // Show messages
    function showMessage(text, success = false) {
        if (!message) return;
        message.textContent = text;
        message.style.color = success ? "green" : "red";
    }
    // Registration
```

```
const registerForm = document.getElementById("registerForm");
if (registerForm) {
    registerForm.addEventListener("submit", async (e) => {
        e.preventDefault();
        const student = {
            name: document.getElementById("name").value.trim(),
            email: document.getElementById("email").value.trim(),
            department: document.getElementById("department").value,
            year: document.getElementById("year").value,
            password: document.getElementById("password").value
        };
        try {
            const response = await fetch("/register", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(student)
            });
            const data = await response.json();
            if (!response.ok) {
                showMessage(data.error || "Registration failed.");
                return;
            }
            showMessage("Registration successful! Redirecting to login...", true);
            registerForm.reset();
            setTimeout(() => {
                window.location.href = "login.html";
            }, 1200);
        } catch (error) {
            showMessage("Server connection failed.");
        }
    });
}
// Login
const loginForm = document.getElementById("loginForm");
if (loginForm) {
    loginForm.addEventListener("submit", async (e) => {
        e.preventDefault();
        const credentials = {
            email: document.getElementById("email").value.trim(),
            password: document.getElementById("password").value
        };
        try {
            const response = await fetch("/login", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(credentials)
            });
            const data = await response.json();
            if (!response.ok) {
                showMessage(data.error || "Login failed.");
                return;
            }
            localStorage.setItem("token", data.token);
            localStorage.setItem("name", data.name);
```

```
                window.location.href = "events.html";
            } catch (error) {
                showMessage("Server connection failed.");
            }
        });
    }
    // All remaining pages require login
    if (page === "events" || page === "my-events") {
        if (!token) {
            window.location.href = "login.html";
            return;
        }
        checkLogin();
    }
    // Verify login and load page
    async function checkLogin() {
        try {
            const response = await fetch("/api/me", {
                headers: {
                    "Authorization": `Bearer ${token}`
                }
            });
            if (!response.ok) {
                localStorage.removeItem("token");
                localStorage.removeItem("name");
                window.location.href = "login.html";
                return;
            }
            const student = await response.json();
            const welcome = document.getElementById("welcome");
            if (welcome) {
                welcome.textContent = "Welcome, " + student.name + "!";
            }
            if (page === "events") {
                loadEvents();
            }
            if (page === "my-events") {
                loadMyEvents();
            }
        } catch (error) {
            showMessage("Could not connect to the server.");
        }
    }
    // Load all events
    async function loadEvents() {
        const container = document.getElementById("eventsContainer");
        showMessage("Loading events...");
        try {
            const response = await fetch("/api/events", {
                headers: {
                    "Authorization": `Bearer ${token}`
                }
            });
            const events = await response.json();
            if (!response.ok) {
                showMessage(events.error || "Could not load events.");
                return;
            }
```

```
            container.innerHTML = "";
            if (events.length === 0) {
                showMessage("No events available right now.");
                return;
            }
            showMessage("");
            events.forEach(event => {
                const card = document.createElement("div");
                card.className = "event-card";
                const category = document.createElement("span");
                category.className = "category";
                category.textContent = event.category;
                const title = document.createElement("h3");
                title.textContent = event.title;
                const description = document.createElement("p");
                description.textContent = event.description;
                const date = document.createElement("p");
                date.textContent = "🗓️ " + event.date;
                const venue = document.createElement("p");
                venue.textContent = "📍 " + event.venue;
                const button = document.createElement("button");
                button.textContent = "Join Event";
                button.addEventListener("click", () => {
                    joinEvent(event._id, button);
                });
                card.append(
                    category,
                    title,
                    description,
                    date,
                    venue,
                    button
                );
                container.appendChild(card);
            });
        } catch (error) {
            showMessage("Could not load events.");
        }
    }
    // Join an event
    async function joinEvent(eventId, button) {
        button.disabled = true;
        button.textContent = "Joining...";
        try {
            const response = await fetch(`/api/events/${eventId}/join`, {
                method: "POST",
                headers: {
                    "Authorization": `Bearer ${token}`
                }
            });
            const data = await response.json();
            if (response.ok) {
                button.textContent = "Joined ✓";
                showMessage(data.message, true);
            } else {
                button.textContent = "Join Event";
                showMessage(data.error || "Could not join event.");
                button.disabled = false;
```

```
            }
        } catch (error) {
            button.textContent = "Join Event";
            button.disabled = false;
            showMessage("Server connection failed.");
        }
    }
    // Load student's joined events
    async function loadMyEvents() {
        const container = document.getElementById("myEventsContainer");
        showMessage("Loading your events...");
        try {
            const response = await fetch("/api/my-events", {
                headers: {
                    "Authorization": `Bearer ${token}`
                }
            });
            const events = await response.json();
            if (!response.ok) {
                showMessage(events.error || "Could not load your events.");
                return;
            }
            container.innerHTML = "";
            if (events.length === 0) {
                showMessage("You haven't joined any events yet.");
                return;
            }
            showMessage("");
            events.forEach(event => {
                const card = document.createElement("div");
                card.className = "event-card";
                const category = document.createElement("span");
                category.className = "category";
                category.textContent = event.category;
                const title = document.createElement("h3");
                title.textContent = event.title;
                const description = document.createElement("p");
                description.textContent = event.description;
                const date = document.createElement("p");
                date.textContent = "📅 " + event.date;
                const venue = document.createElement("p");
                venue.textContent = "📍 " + event.venue;
                const status = document.createElement("p");
                status.textContent = "✓ Successfully Registered";
                status.style.color = "green";
                card.append(category, title, description, date, venue, status);
                container.appendChild(card);
            });
        } catch (error) {
            showMessage("Could not load your registered events.");
        }
    }
    // Logout
    const logoutBtn = document.getElementById("logoutBtn");
    if (logoutBtn) {
        logoutBtn.addEventListener("click", () => {
            localStorage.removeItem("token");
            localStorage.removeItem("name");
```

```
            window.location.href = "login.html";
        });
    }
});
```

### 4.1.4 NODE.JS CODE

The Node.js back-end processes registration and login requests. It checks the submitted information, hashes passwords and generates authentication tokens.

**server.js**

```
const express = require("express");
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const dotenv = require("dotenv");
const path = require("path");
dotenv.config();
const app = express();
const PORT = process.env.PORT || 3000;
const JWT_SECRET = process.env.JWT_SECRET;
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));
// Check environment variables
if (!process.env.MONGODB_URI || !JWT_SECRET) {
   throw new Error("Please check your .env file.");
}
// Homepage
app.get("/", (req, res) => {
   res.redirect("/register.html");
});
// MongoDB Connection
mongoose.connect(process.env.MONGODB_URI)
   .then(() => console.log("MongoDB Connected Successfully"))
   .catch(error => console.log("MongoDB Error:", error.message));
// Student Schema
const studentSchema = new mongoose.Schema({
   name: { type: String, required: true },
   email: { type: String, required: true, unique: true },
   department: { type: String, required: true },
   year: { type: String, required: true },
   password: { type: String, required: true }
});
const Student = mongoose.model("Student", studentSchema);
```

```
// Event Schema
const eventSchema = new mongoose.Schema({
    title: String,
    description: String,
    date: String,
    venue: String,
    category: String
});
const Event = mongoose.model("Event", eventSchema);
// Event Registration Schema
const registrationSchema = new mongoose.Schema({
    student: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Student",
        required: true
    },
    event: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Event",
        required: true
    }
});

registrationSchema.index(
    { student: 1, event: 1 },
    { unique: true }
);
const Registration = mongoose.model("Registration", registrationSchema);
// Authentication Middleware
function authenticateToken(req, res, next) {
    const authHeader = req.headers.authorization;
    const token = authHeader && authHeader.split(" ")[1];
    if (!token) {
        return res.status(401).json({
            error: "Please login first."
        });
    }

    jwt.verify(token, JWT_SECRET, (error, user) => {
```

```
        if (error) {
            return res.status(403).json({
                error: "Invalid or expired login."
            });
        }
        req.user = user;
        next();
    });
}
// Student Registration
app.post("/register", async (req, res) => {
    try {
        const { name, email, department, year, password } = req.body;
        if (!name || !email || !department || !year || !password) {
            return res.status(400).json({
                error: "Please fill in all fields."
            });
        }
        if (password.length < 8) {
            return res.status(400).json({
                error: "Password must contain at least 8 characters."
            });
        }
        const normalizedEmail = email.toLowerCase().trim();
        const existingStudent = await Student.findOne({
            email: normalizedEmail
        });
        if (existingStudent) {
            return res.status(409).json({
                error: "Email already registered."
            });
        }
        const hashedPassword = await bcrypt.hash(password, 10);
        const student = new Student({
            name: name.trim(),
            email: normalizedEmail,
            department,
            year,
            password: hashedPassword
```

```
        });
        await student.save();

        res.status(201).json({
            message: "Registration successful!"
        });
    } catch (error) {
        console.log("Registration Error:", error.message);
        res.status(500).json({
            error: "Registration failed."
        });
    }
});
// Student Login
app.post("/login", async (req, res) => {
    try {
        const { email, password } = req.body;
        const student = await Student.findOne({
            email: (email || "").toLowerCase().trim()
        });
        if (!student || !(await bcrypt.compare(password || "", student.password))) {
            return res.status(401).json({
                error: "Invalid email or password."
            });
        }
        const token = jwt.sign(
            { id: student._id },
            JWT_SECRET,
            { expiresIn: "1d" }
        );

        res.json({
            message: "Login successful!",
            token,
            name: student.name
        });
    } catch (error) {
        console.log("Login Error:", error.message);
        res.status(500).json({
```

```
            error: "Login failed."
        });
    }
});
// Get Student Details
app.get("/api/me", authenticateToken, async (req, res) => {
    try {
        const student = await Student.findById(req.user.id)
            .select("name email department year");
        if (!student) {
            return res.status(404).json({
                error: "Student not found."
            });
        }
        res.json(student);
    } catch (error) {
        res.status(500).json({
            error: "Could not load student details."
        });
    }
});
// Add Sample Events
async function addSampleEvents() {
    const count = await Event.countDocuments();
    if (count === 0) {
        await Event.insertMany([
            {
                title: "TechFest 2026",
                description: "Explore technology, coding competitions and exciting innovations.",
                date: "2026-12-10",
                venue: "Main Auditorium",
                category: "Technical"
            },
            {
                title: "College Hackathon",
                description: "Build creative solutions and showcase your coding skills.",
                date: "2026-12-15",
                venue: "Computer Science Block",
                category: "Technical"
```

```
        },
        {
          title: "Cultural Fest",
          description: "Celebrate music, dance, art and college talent.",
          date: "2026-12-20",
          venue: "College Ground",
          category: "Cultural"
        },
        {
          title: "Photography Contest",
          description: "Capture memorable moments and share your creativity.",
          date: "2026-12-22",
          venue: "Seminar Hall",
          category: "Arts"
        },
        {
          title: "Sports Day",
          description: "Participate in exciting indoor and outdoor sports.",
          date: "2027-01-05",
          venue: "Sports Ground",
          category: "Sports"
        },
        {
          title: "Dance Competition",
          description: "Show your moves in an energetic dance competition.",
          date: "2027-01-10",
          venue: "Open Stage",
          category: "Cultural"
        }
      ]);
      console.log("Sample events added successfully.");
  }
}
// Get All Events
app.get("/api/events", authenticateToken, async (req, res) => {
  try {
    const events = await Event.find().sort({ date: 1 });
    res.json(events);
  } catch (error) {
```

```
        res.status(500).json({
            error: "Could not load events."
        });
    }
});
// Join an Event
app.post("/api/events/:id/join", authenticateToken, async (req, res) => {
    try {
        const event = await Event.findById(req.params.id);
        if (!event) {
            return res.status(404).json({
                error: "Event not found."
            });
        }
        const existing = await Registration.findOne({
            student: req.user.id,
            event: event._id
        });
        if (existing) {
            return res.status(409).json({
                error: "You have already joined this event."
            });
        }
        await Registration.create({
            student: req.user.id,
            event: event._id
        });

        res.status(201).json({
            message: "Successfully joined " + event.title + "!"
        });
    } catch (error) {
        if (error.code === 11000) {
            return res.status(409).json({
                error: "You have already joined this event."
            });
        }

        res.status(500).json({
```

```
            error: "Could not join event."
        });
    }
});
// Get My Registered Events
app.get("/api/my-events", authenticateToken, async (req, res) => {
    try {
        const registrations = await Registration.find({
            student: req.user.id
        }).populate("event");
        const events = registrations
            .filter(item => item.event)
            .map(item => item.event);
        res.json(events);
    } catch (error) {
        res.status(500).json({
            error: "Could not load your events."
        });
    }
});
// Start Server
async function startServer() {
    try {
        await mongoose.connection.asPromise();
        await addSampleEvents();
        app.listen(PORT, () => {
            console.log("MongoDB Connected Successfully");
            console.log("College Event Management running at http://localhost:" + PORT);
        });
    } catch (error) {
        console.log("Server startup error:", error.message);
        process.exit(1);
    }
}
startServer();
```

## package.json (PROJECT DEPENDENCIES)

The package.json file lists the project metadata and the npm packages used by the server (Express.js, Mongoose, bcryptjs and jsonwebtoken, with dotenv for environment variables). It is placed in the project root, next to server.js:

```
{
```

```
  "name": "college-event-management",
  "version": "1.0.0",
  "description": "College Event Management System",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "dotenv": "^18.0.9",
    "express": "^5.3.0",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.11.4"
  }
}
```

## .env FILE (ENVIRONMENT VARIABLES)

The server loads its configuration from a .env file in the project root using dotenv. The file contains only two values — the MongoDB connection string and the JWT secret key — and is not uploaded to any public repository:

```
MONGODB_URI=mongodb://127.0.0.1:27017/collegeEventDB
JWT_SECRET=<your-secret-key>
```

Note: MONGODB_URI tells the server which MongoDB database to connect to (collegeEventDB on the local machine). JWT_SECRET is the secret key used to sign and verify the JWT login tokens. The actual secret value is kept private and is not printed in this report — replace <your-secret-key> with your own long random value in your local .env file.

### 4.1.5 OUTPUT SCREENSHOT

**Figure 4.1.1: Student Registration Page (filled form)**

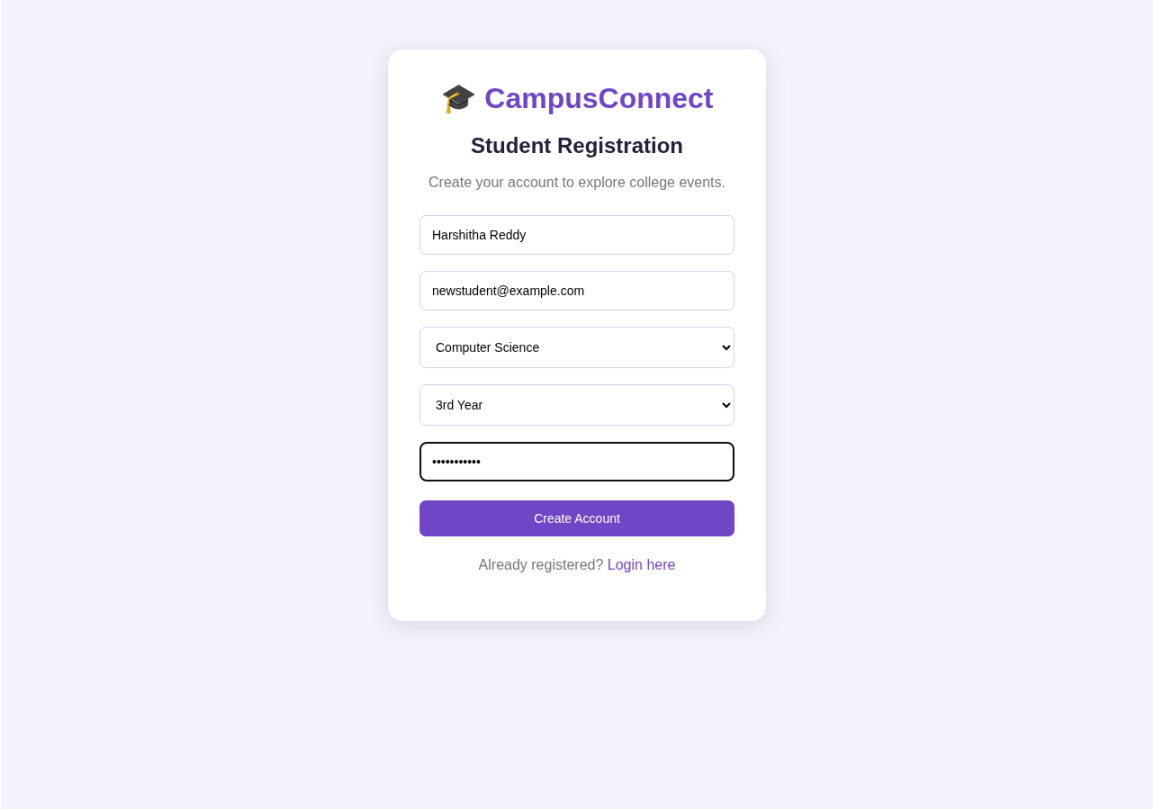
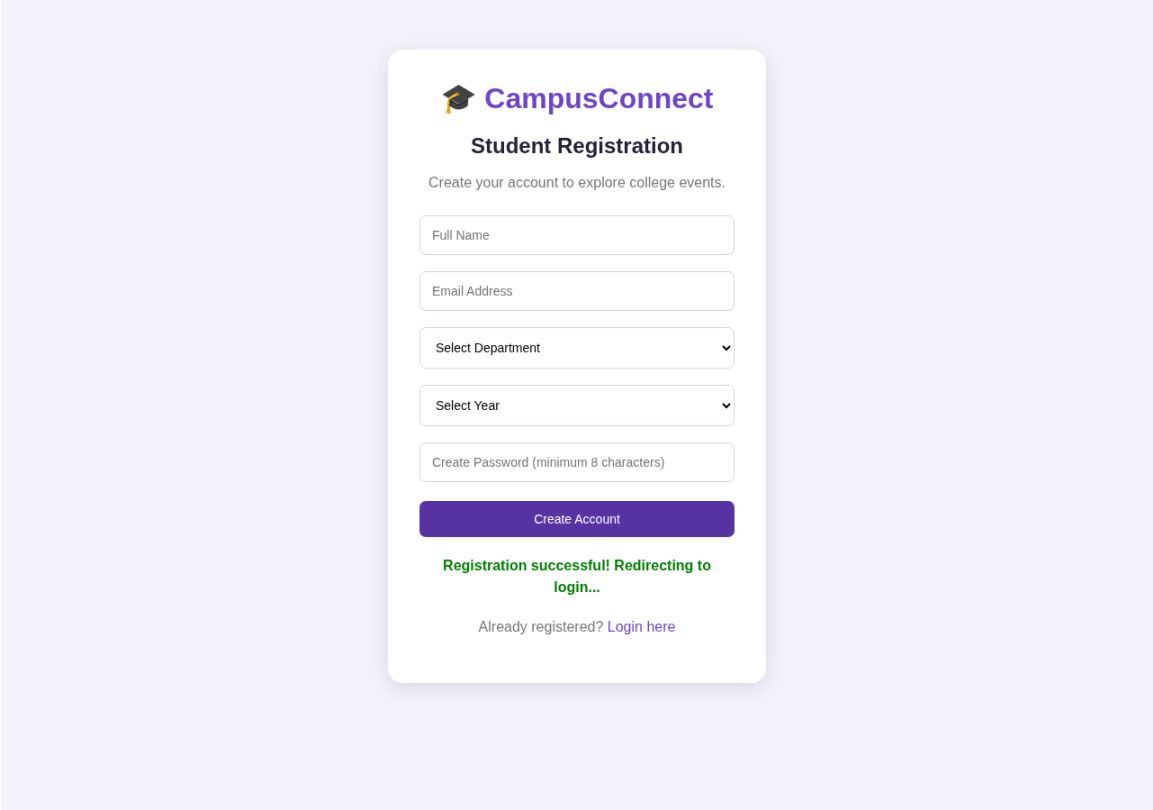
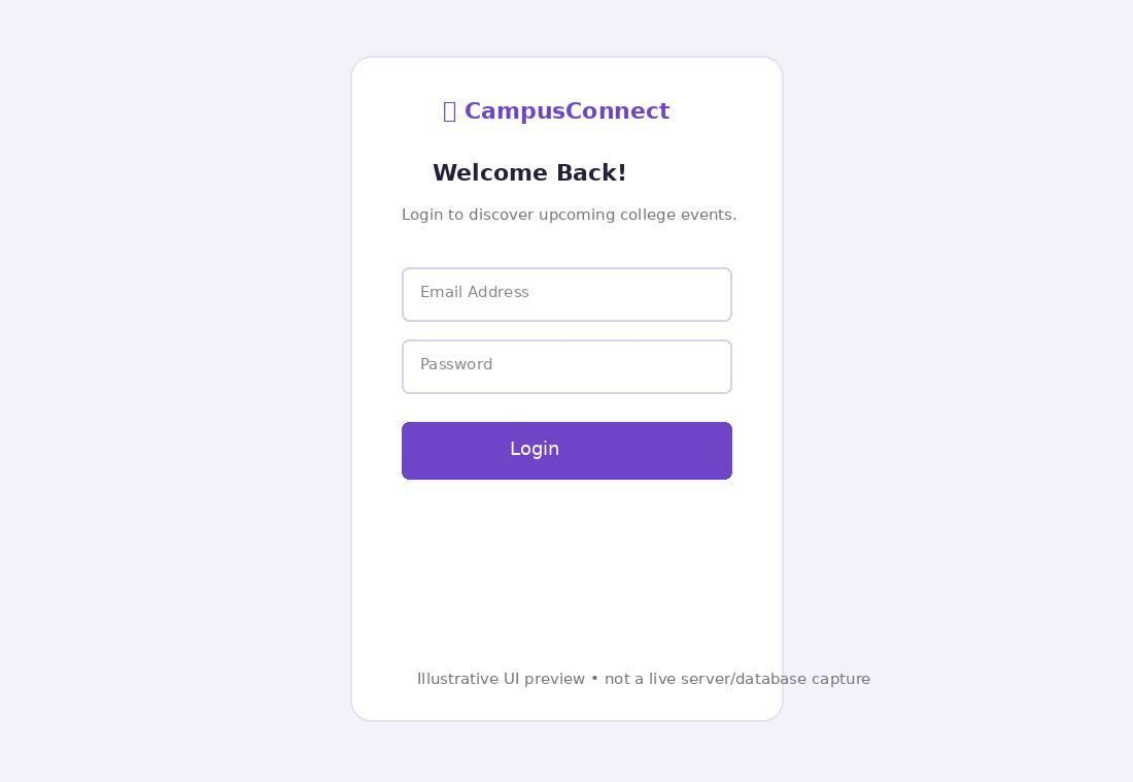
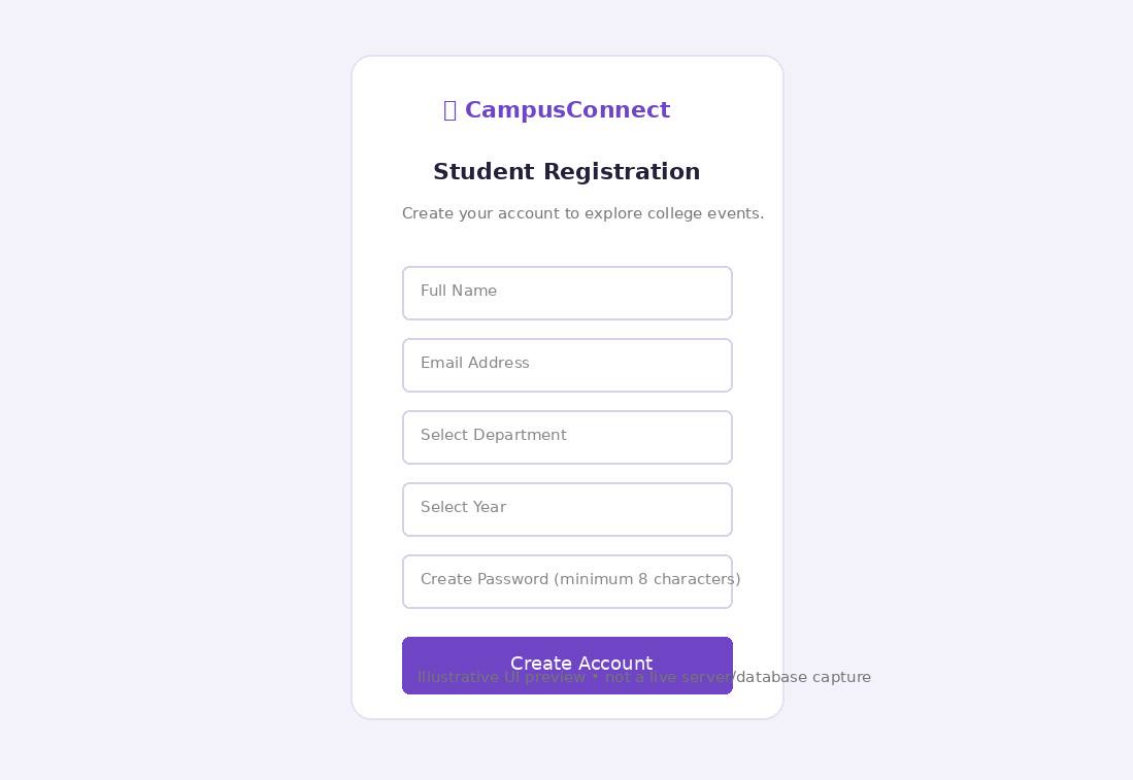



**Figure 4.1.2: Registration Successful — redirecting to login**






## 4.2 EVENT DISPLAY MODULE

The Event Display Module allows students to view all available college events. Event details such as title, description, category, venue, and date are retrieved from MongoDB and displayed using event cards. The module helps students explore and select events of interest.

### 4.2.1 HTML CODE

The event display page is created using HTML and contains navigation menus, event containers, and sections for displaying event information dynamically.

events.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Events</title>
  <link rel="stylesheet" href="style.css">
</head>
<body data-page="events">
  <nav>
    <h2>🎓 CampusConnect</h2>
    <div>
      <a href="events.html">Events</a>
      <a href="my-events.html">My Events</a>
      <button id="logoutBtn">Logout</button>
    </div>
  </nav>
  <header class="hero">
    <h1>Discover Campus Events</h1>
    <p>Explore, participate and make unforgettable memories!</p>
    <h3 id="welcome"></h3>
  </header>
  <main class="main-container">
    <h2>Upcoming Events</h2>
    <div id="eventsContainer" class="event-grid"></div>
    <p id="message"></p>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

### 4.2.2 CSS CODE

CSS is used to design event cards, navigation bars, page layouts, and responsive interfaces.

Insert the relevant code from:

Note: style.css is a single shared file used by the whole application. It is already given in full in Section 4.1.2 and is not repeated here. The css behaviour described above for this module is implemented by that shared file.

### 4.2.3 JAVASCRIPT CODE

JavaScript retrieves event information from the server and dynamically displays it on the page.

Note: script.js is a single shared file used by the whole application. It is already given in full in Section 4.1.3 and is not repeated here. The javascript behaviour described above for this module is implemented by that shared file.

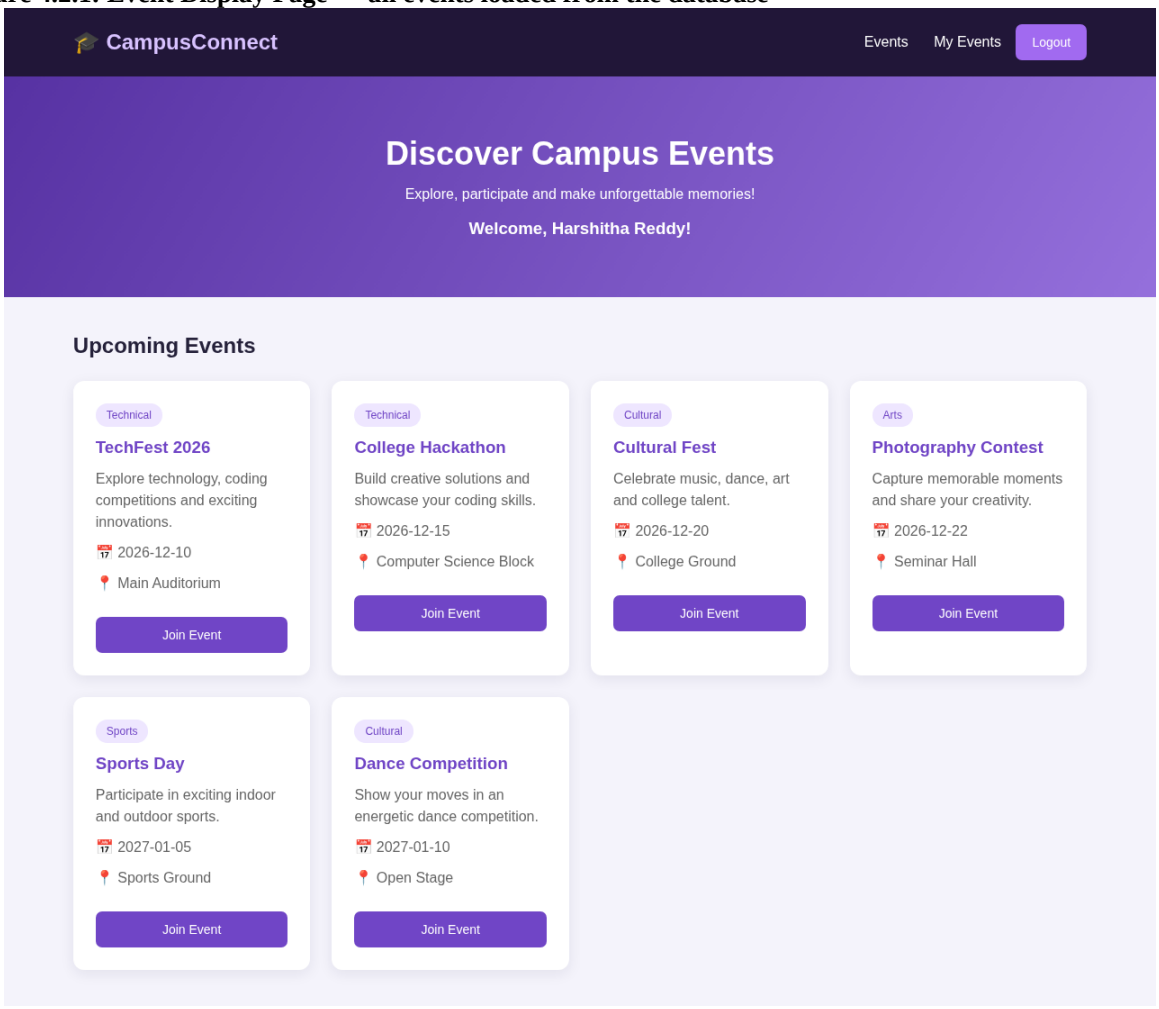
### 4.2.4 NODE.JS CODE

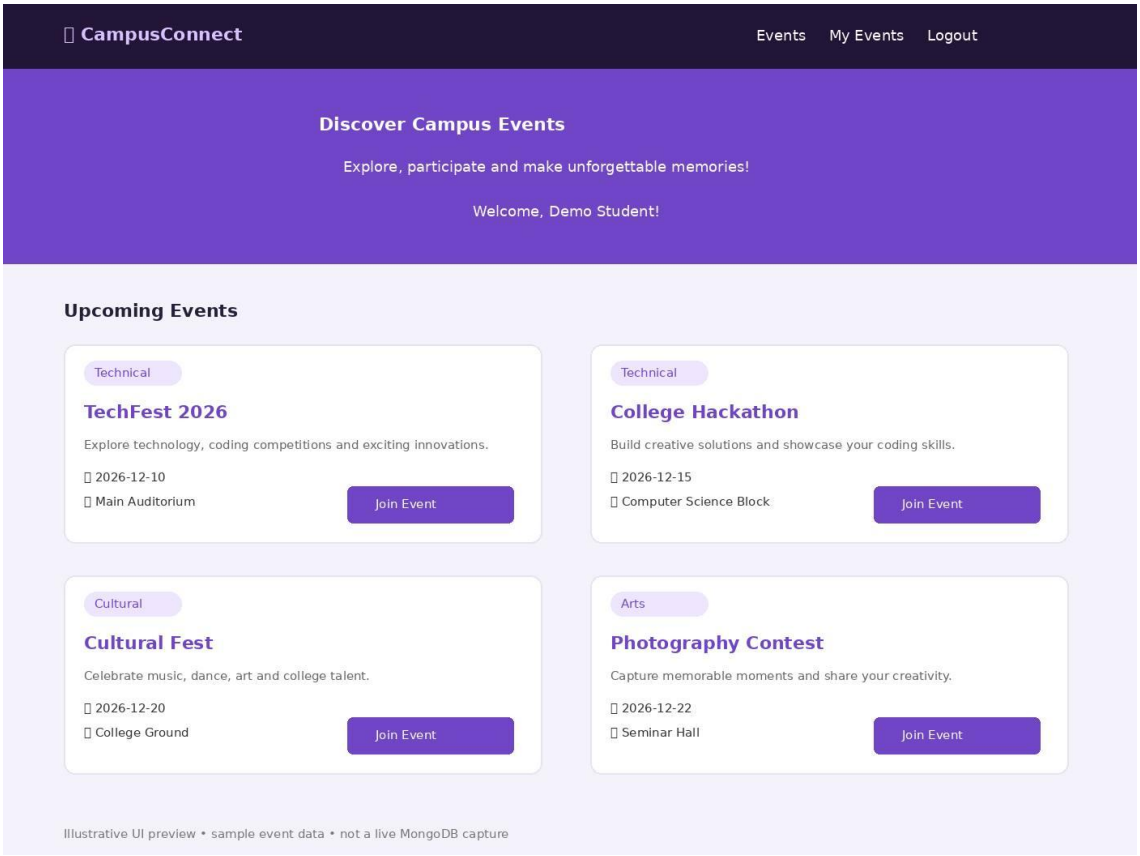
Node.js retrieves event information from MongoDB and provides API endpoints for displaying available events.

Note: server.js is a single shared file used by the whole application. It is already given in full in Section 4.1.4 and is not repeated here. The node.js behaviour described above for this module is implemented by that shared file.

### 4.2.5 OUTPUT SCREENSHOT

**Figure 4.2.1: Event Display Page — all events loaded from the database**






## 4.3 EVENT REGISTRATION MODULE

The Event Registration Module allows students to join events through an online registration process. Students can register for available events using a simple interface. The system prevents duplicate registrations and stores registration details securely.

### 4.3.1 HTML CODE

The event registration interface is developed using HTML and integrated into the event display page.

**events.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Events</title>
  <link rel="stylesheet" href="style.css">
</head>
<body data-page="events">
  <nav>
    <h2>🎓 CampusConnect</h2>
    <div>
      <a href="events.html">Events</a>
```

```
        <a href="my-events.html">My Events</a>
        <button id="logoutBtn">Logout</button>
      </div>
    </nav>
    <header class="hero">
      <h1>Discover Campus Events</h1>
      <p>Explore, participate and make unforgettable memories!</p>
      <h3 id="welcome"></h3>
    </header>
    <main class="main-container">
      <h2>Upcoming Events</h2>
      <div id="eventsContainer" class="event-grid"></div>
      <p id="message"></p>
    </main>
    <script src="script.js"></script>
</body>
</html>
```

### 4.3.2 CSS CODE

CSS is used to style event cards, registration buttons, and user interfaces.

Note: style.css is a single shared file used by the whole application. It is already given in full in Section 4.1.2 and is not repeated here. The css behaviour described above for this module is implemented by that shared file.

### 4.3.3 JAVASCRIPT CODE

JavaScript handles event registration requests and communicates with the server using APIs.

Note: script.js is a single shared file used by the whole application. It is already given in full in Section 4.1.3 and is not repeated here. The javascript behaviour described above for this module is implemented by that shared file.

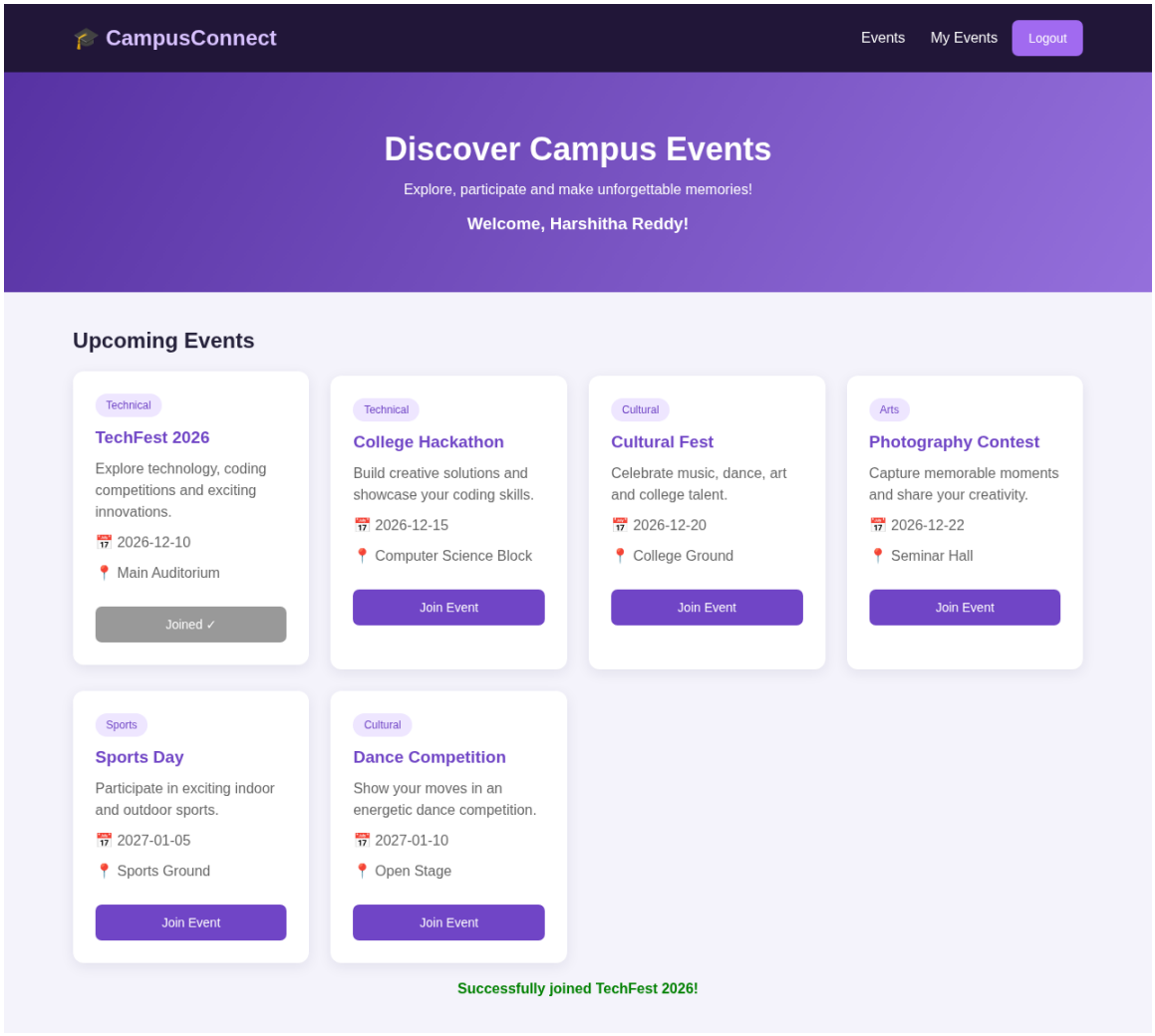
### 4.3.4 NODE.JS CODE

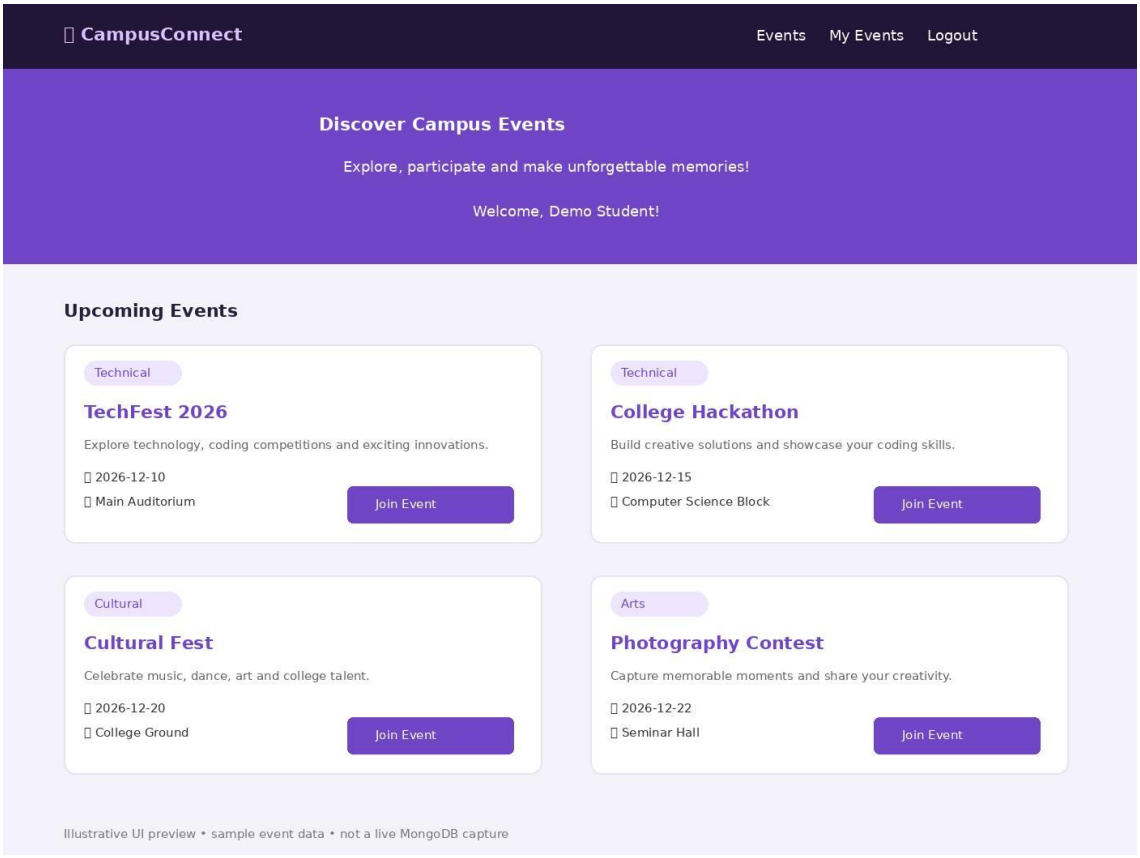
The back-end processes event registration requests and stores registration details in MongoDB.

Note: server.js is a single shared file used by the whole application. It is already given in full in Section 4.1.4 and is not repeated here. The node.js behaviour described above for this module is implemented by that shared file.

### 4.3.5 OUTPUT SCREENSHOT

**Figure 4.3.1: Event Registration — "Successfully joined TechFest 2026!"**


CampusConnect
Events
My Events
Logout
Discover Campus Events
Explore, participate and make unforgettable memories!
Welcome, Harshitha Reddy!
Upcoming Events
Technical
TechFest 2026
Explore technology, coding competitions and exciting innovations.
2026-12-10
Main Auditorium
Joined ✓
Technical
College Hackathon
Build creative solutions and showcase your coding skills.
2026-12-15
Computer Science Block
Join Event
Cultural
Cultural Fest
Celebrate music, dance, art and college talent.
2026-12-20
College Ground
Join Event
Arts
Photography Contest
Capture memorable moments and share your creativity.
2026-12-22
Seminar Hall
Join Event
Sports
Sports Day
Participate in exciting indoor and outdoor sports.
2027-01-05
Sports Ground
Join Event
Cultural
Dance Competition
Show your moves in an energetic dance competition.
2027-01-10
Open Stage
Join Event
Successfully joined TechFest 2026!




## 4.4 MY EVENTS MODULE

The My Events Module enables students to view all events that they have joined. Registered events are displayed in a dedicated section, helping students track their participation.

### 4.4.1 HTML CODE

The My Events page is developed using HTML and provides a dedicated area for displaying registered events.

**my-events.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Events</title>
  <link rel="stylesheet" href="style.css">
</head>
<body data-page="my-events">
  <nav>
    <h2>🎓 CampusConnect</h2>
    <div>
      <a href="events.html">Events</a>
```

```
        <a href="my-events.html">My Events</a>
        <button id="logoutBtn">Logout</button>
      </div>
  </nav>
<header class="hero">
      <h1>My Registered Events</h1>
      <p>All the events you have joined in one place.</p>
  </header>
<main class="main-container">
      <div id="myEventsContainer" class="event-grid"></div>
      <p id="message"></p></main>
<script src="script.js"></script>
</body></html>
```

### 4.4.2 CSS CODE

CSS is used to style event cards and the My Events interface.

Note: style.css is a single shared file used by the whole application. It is already given in full in Section 4.1.2 and is not repeated here. The css behaviour described above for this module is implemented by that shared file.

### 4.4.3 JAVASCRIPT CODE

JavaScript retrieves registered event details from the server and displays them dynamically.

Note: script.js is a single shared file used by the whole application. It is already given in full in Section 4.1.3 and is not repeated here. The javascript behaviour described above for this module is implemented by that shared file.

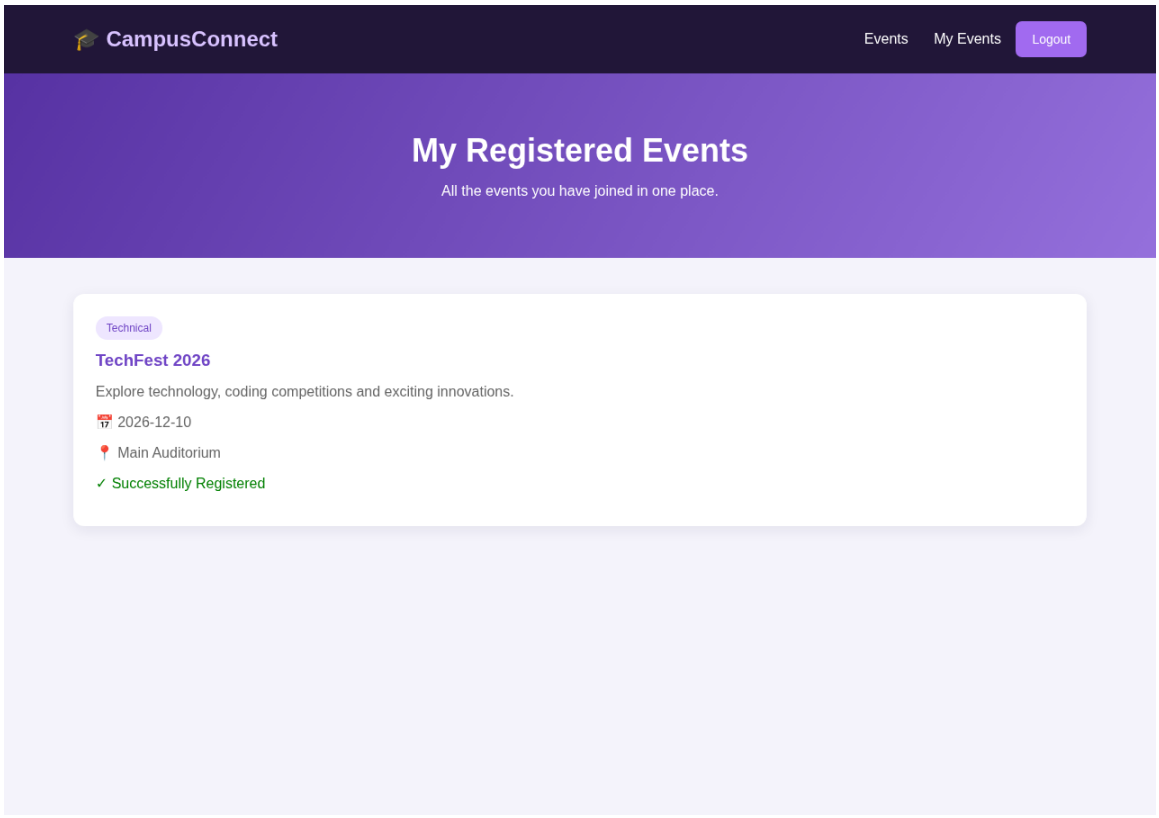
### 4.4.4 NODE.JS CODE

The back-end provides API endpoints to retrieve all events registered by a student.

Note: server.js is a single shared file used by the whole application. It is already given in full in Section 4.1.4 and is not repeated here. The node.js behaviour described above for this module is implemented by that shared file.

### 4.4.5 OUTPUT SCREENSHOT

**Figure 4.4.1: My Events Page — registered events with “Successfully Registered” status**




## 5.0 DATABASE

The College Event Management System uses MongoDB as its database. MongoDB stores data in collections using document-based records. The application contains three major collections: Students, Events, and Registrations.

### 5.1 DATABASE DESIGN (TABLE STRUCTURE)

**STUDENT COLLECTION**

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique Student ID |
| name | String | Student Name |
| email | String | Student Email |
| department | String | Department Name |
| year | String | Study Year |
| password | String | Encrypted Password |

**EVENT COLLECTION**

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique Event ID |

| | | |
|---|---|---|
| title | String | Event Title |
| description | String | Event Description |
| date | String | Event Date |
| venue | String | Event Venue |
| category | String | Event Category |

**REGISTRATION COLLECTION**

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique Registration ID |
| student | ObjectId | Reference to Student |
| event | ObjectId | Reference to Event |

## DATABASE CONNECTION — .env FILE

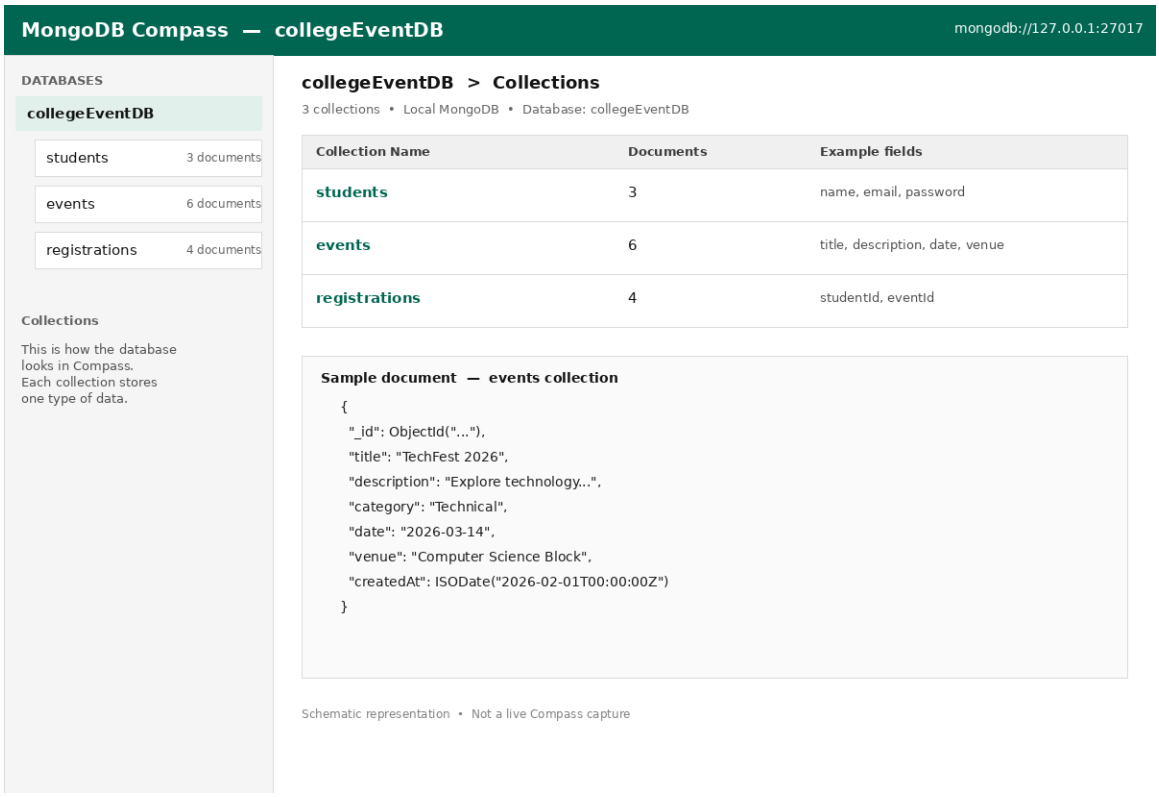
The database connection in server.js uses the values from the .env file, loaded with dotenv. The same .env file shown in Section 4.1.4 is used here:

```
MONGODB_URI=mongodb://127.0.0.1:27017/collegeEventDB
JWT_SECRET=<your-secret-key>
```

Note: MONGODB_URI tells the server which MongoDB database to connect to (collegeEventDB on the local machine). JWT_SECRET is the secret key used to sign and verify the JWT login tokens. The actual secret value is kept private and is not printed in this report — replace <your-secret-key> with your own long random value in your local .env file.

5.2 DATABASE OUTPUT SCREENSHOT

**Figure 5.1: Database Output — schematic representation of the students, events and registrations collections in collegeEventDB (structure shown as stored in MongoDB)**




## 6.0 VALIDATION

The validation module ensures that the data entered by users is accurate, secure, and complete. Validation is implemented on both the client side and server side to prevent invalid data entry and unauthorized access.

### 6.1 FORM VALIDATION (FRONT-END)

Front-end validation is implemented using HTML form attributes and JavaScript.

The system checks that:

- Required fields are not left empty.
- Email addresses follow the correct format.
- Passwords contain a minimum of 8 characters.
- Department and year fields are selected.
- User inputs are submitted through the appropriate forms.

### 6.2 BACKEND VALIDATION (NODE.JS)

Back-end validation is implemented using Express.js and MongoDB.

The system checks that:

- All registration fields are provided.
- Email addresses are unique.
- Passwords satisfy the minimum length requirement.
- Users provide valid login credentials.

- JWT tokens are verified before accessing protected pages.
- Students cannot register multiple times for the same event.

## 6.3 VALIDATION OUTPUT SCREENSHOT

**Figure 6.1: Empty Registration Form Validation — browser requires all fields**

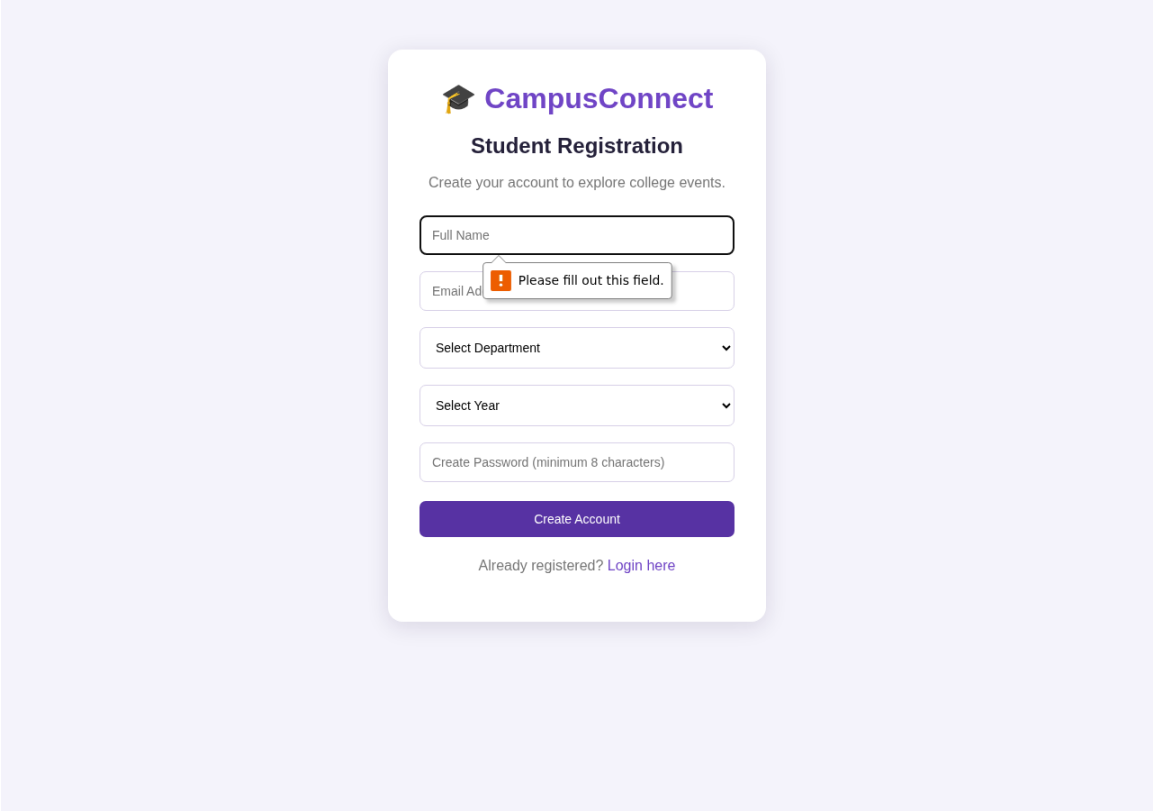



**Figure 6.2: Invalid Login Message — "Invalid email or password."**

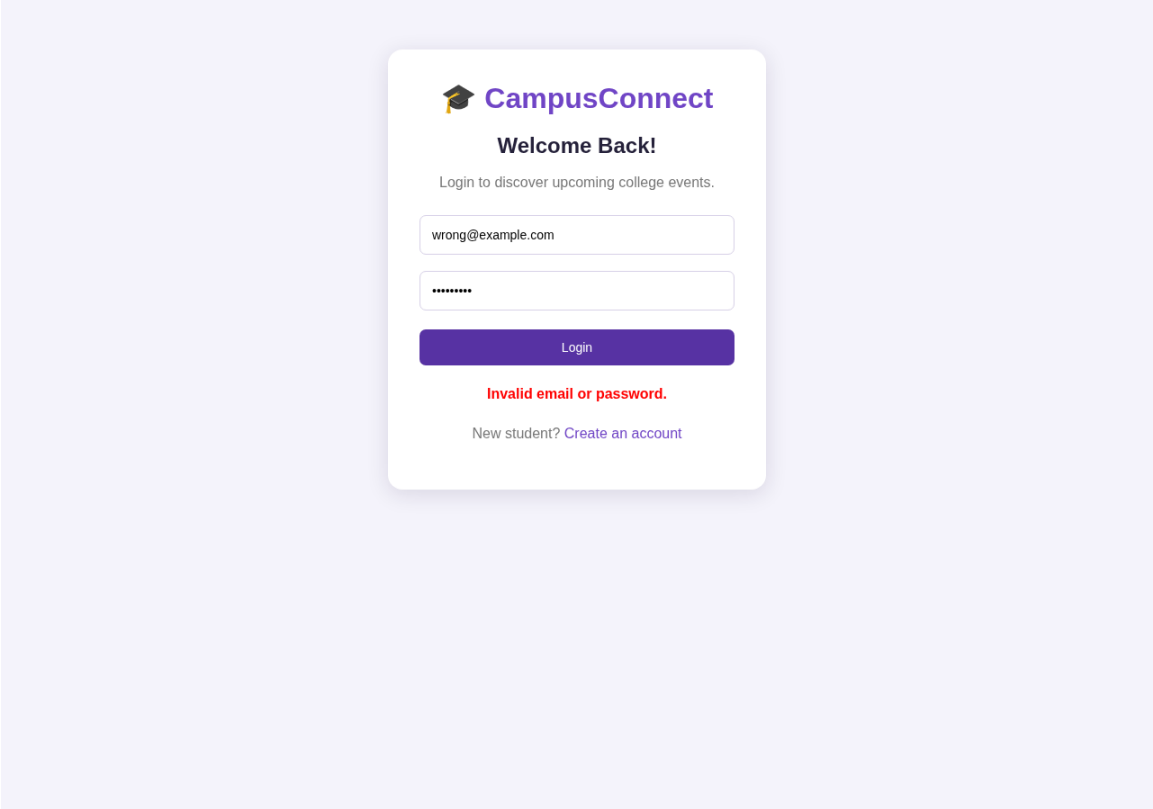



**Figure 6.3: Duplicate Email Registration Message — "Email already registered."**

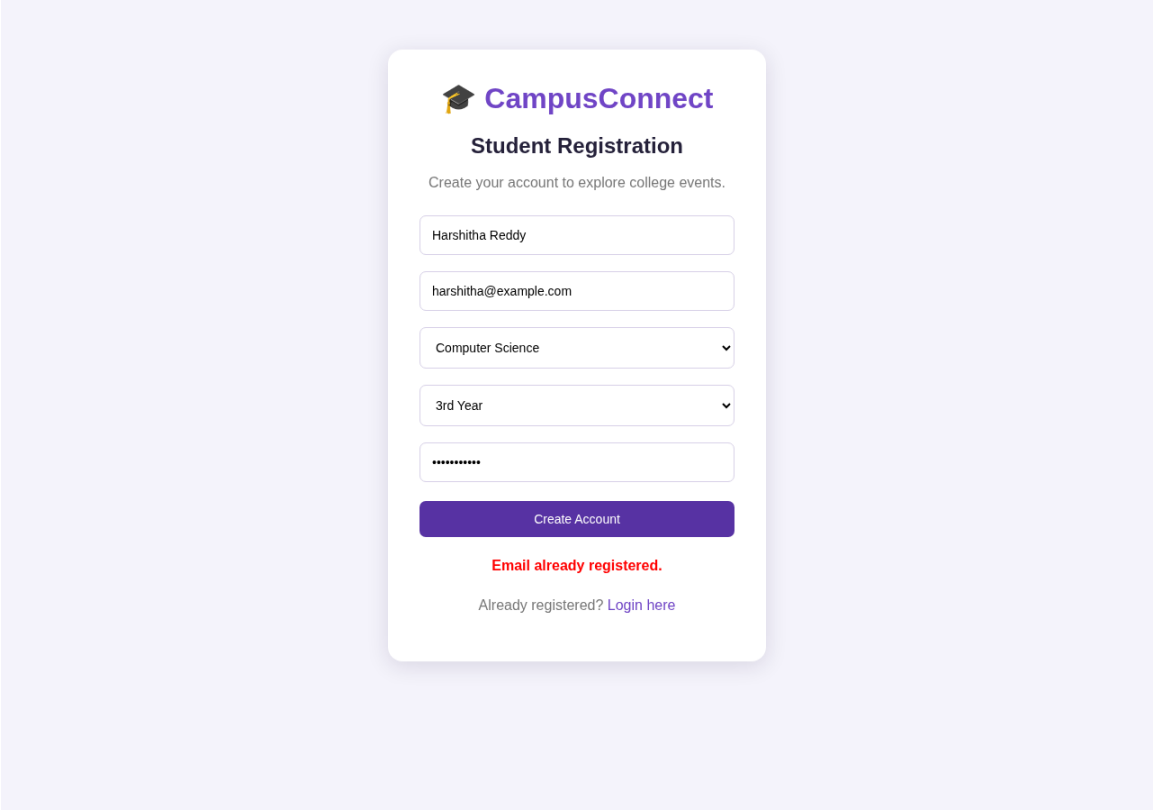



**Figure 6.4: Duplicate Event Registration Message — "You have already joined this event."**

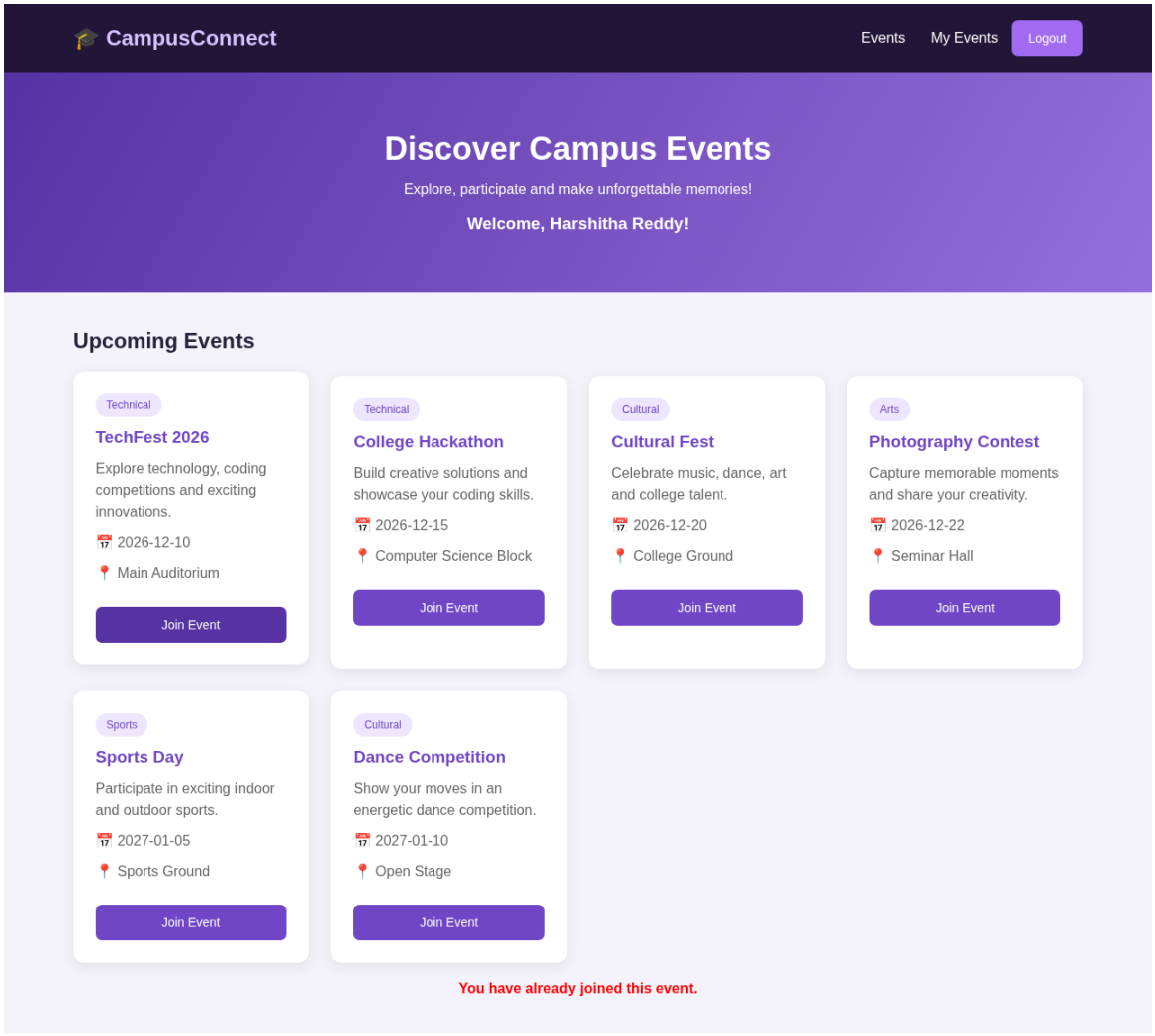
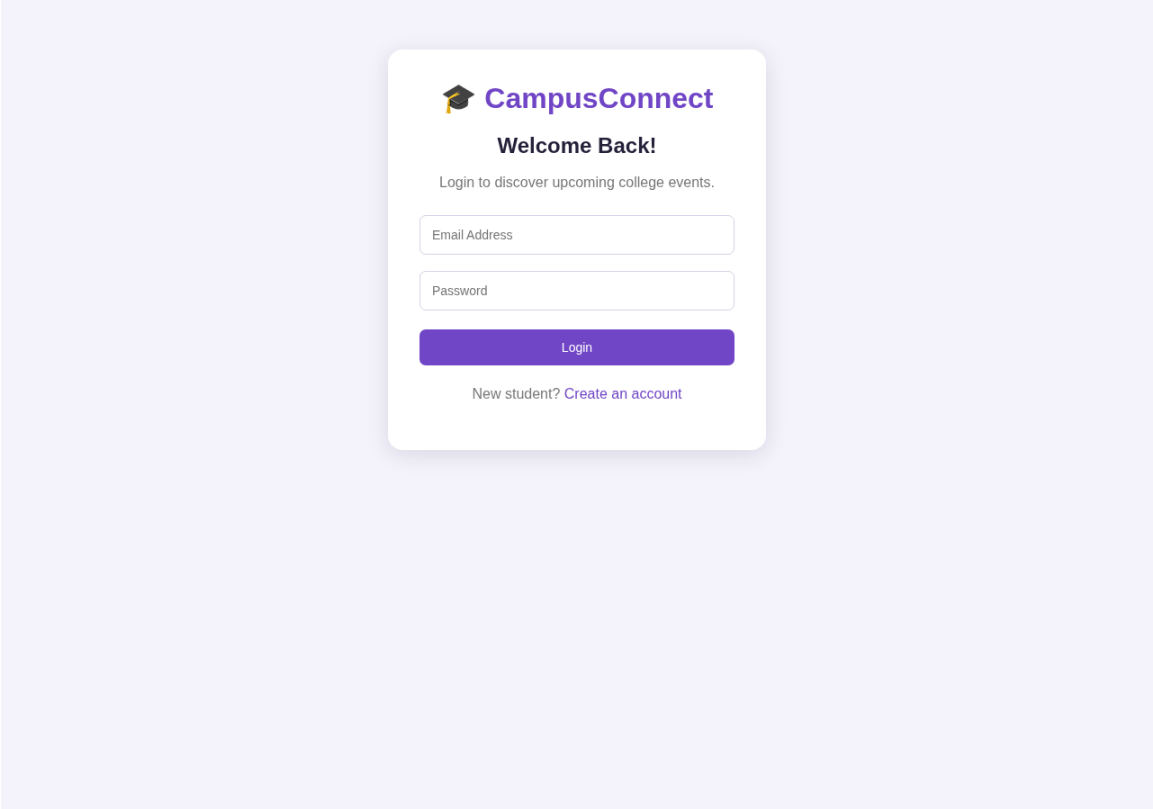



**Figure 6.5: Unauthorized Access Restriction — events page without login redirects to Login**




1.
2.
3.

In Microsoft Word, Ctrl + Click on the link above to open the repository page in your browser.

**GitHub Repository:** https://github.com/Harshitha0-07/College-Event-Management

The complete source code for this project (HTML, CSS, JavaScript, Node.js / Express.js, MongoDB configuration and this project report) is available in the GitHub repository below:

## PROJECT SOURCE CODE — GITHUB REPOSITORY

### 7.0 CONCLUSION

The **College Event Management System** was developed to simplify the process of organizing and participating in college events through a web-based application. The project provides features such as student registration, login, event display, event participation, and viewing registered events.

The system uses HTML, CSS, and JavaScript for creating an interactive user interface, while Node.js and Express.js provide efficient server-side functionality. MongoDB is used to store event and student data securely.

Overall, the project demonstrates how modern web technologies can be combined to develop an organized and efficient platform for managing college events and improving student participation.

## 8.0 REFERENCES


1. HTML Documentation – https://developer.mozilla.org/en-US/docs/Web/HTML
2. CSS Documentation – https://developer.mozilla.org/en-US/docs/Web/CSS
3. JavaScript Documentation – https://developer.mozilla.org/en-US/docs/Web/JavaScript
4. Node.js Documentation – https://nodejs.org/docs/
5. Express.js Documentation – https://expressjs.com/
6. MongoDB Documentation – https://www.mongodb.com/docs/
7. Mongoose Documentation – https://mongoosejs.com/docs/
8. JSON Web Token Documentation – https://jwt.io/introduction
9. bcryptjs Package Documentation – https://www.npmjs.com/package/bcryptjs